

# EE200: Signals, Systems and Networks

## Course Project

### QUESTION 3 REPORT

TEAM MEMBERS:

**240622 Manish Kajla**

**241114 Utkarsh Aman**

### Abstract

*This project explores the development of a Shazam-inspired audio recognition system based on spectrogram analysis and audio fingerprinting. The objective is to identify songs from short and potentially noisy audio clips by extracting distinctive frequency-time characteristics rather than comparing raw audio signals directly. The process begins with transforming audio recordings into spectrograms using the Short-Time Fourier Transform (STFT), providing a time-varying representation of frequency content. Prominent spectral peaks are then detected and represented as constellation maps, capturing the most significant acoustic features of a song. These peaks are paired to generate compact fingerprints that remain robust under noise and minor distortions. A database of fingerprints is constructed for a collection of songs, and unknown audio clips are identified by matching their fingerprints against the stored database and analyzing offset consistency. The project further investigates the impact of noise, time shifts, and spectrogram parameter choices on recognition performance, highlighting the trade-off between time and frequency resolution. Finally, the fingerprinting pipeline is integrated into a simple software application capable of indexing songs and recognizing query clips. The results demonstrate how sparse spectral features can provide accurate, efficient, and noise-resistant audio identification, illustrating the core principles behind modern music recognition systems.*

**Live Streamlit App:**

<https://audio-fingerprinting-241114-240622.streamlit.app/>

**Alternate Streamlit App (if the above link is unavailable):**

<https://audio-fingerprinting-240622-241114.streamlit.app/>

**Source Code Repository:**

<https://github.com/Utkarsh-Aman/Audio-Fingerprinting>

## Contents

|          |                                                         |          |
|----------|---------------------------------------------------------|----------|
| <b>1</b> | <b>1. The Limitation of a Single Fourier Transform</b>  | <b>3</b> |
| <b>2</b> | <b>2. The Spectrogram and Time-Frequency Resolution</b> | <b>3</b> |
| <b>3</b> | <b>3. Constellation Map (Peak Picking)</b>              | <b>4</b> |
| <b>4</b> | <b>4. Hash Generation and Pairing</b>                   | <b>5</b> |
| <b>5</b> | <b>5. Identification and Offset Matching</b>            | <b>5</b> |
| <b>6</b> | <b>6. Paired Hashes vs. Single Peaks</b>                | <b>5</b> |
| <b>7</b> | <b>7. Robustness Testing</b>                            | <b>6</b> |
| 7.1      | a. Noise Robustness . . . . .                           | 6        |
| 7.2      | b. Pitch Shift Robustness . . . . .                     | 6        |
| 7.3      | c. Proposed Improvements . . . . .                      | 7        |

## 1 1. The Limitation of a Single Fourier Transform

To understand why a single Fourier transform is insufficient for audio fingerprinting, we computed the Discrete Fourier Transform (DFT) magnitude of an entire song ("Yesterday"). As shown in Figure 1, the DFT provides a global frequency profile, revealing which frequencies are present overall. However, all sense of timing is completely lost. A note played at the beginning of the song and a note played at the end are indistinguishable in this global spectrum. For fingerprinting, we need to know *when* specific frequencies occur.

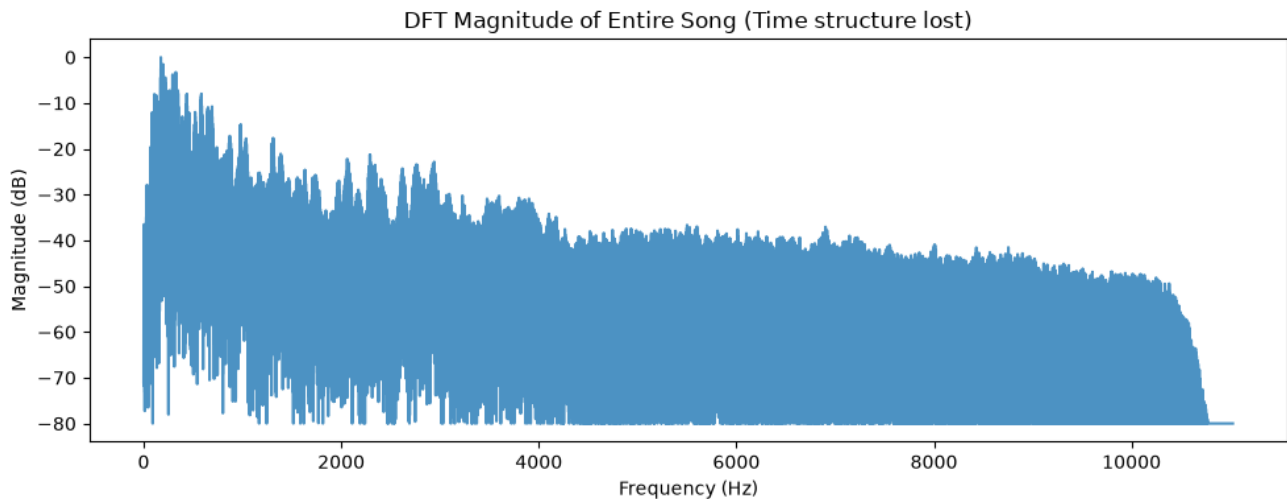


Figure 1: DFT Magnitude of an entire song. The time axis is collapsed.

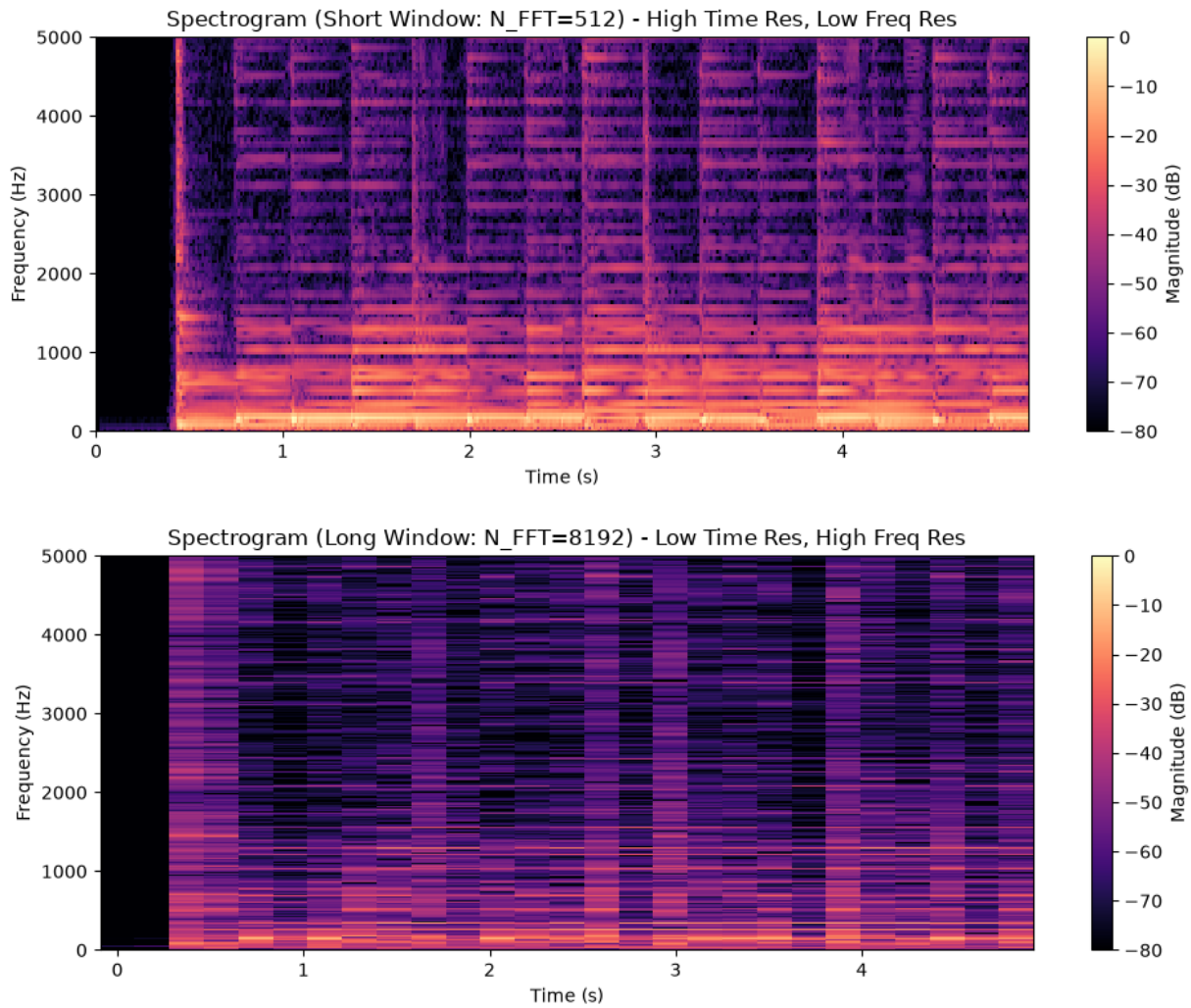
## 2 2. The Spectrogram and Time-Frequency Resolution

To capture both time and frequency information, we use a spectrogram. By sliding a short window across the signal, computing the DFT for each slice, and stacking the results, we can watch frequency content evolve over time.

We compared spectrograms computed with a short window ( $N_{FFT} = 512$ ) and a long window ( $N_{FFT} = 8192$ ), shown in Figure 2. This demonstrates the fundamental time-frequency tradeoff (the uncertainty principle):

- **Short Window:** Provides excellent time resolution, allowing us to pinpoint fast transient events (like drum hits) sharply in time. However, frequency bins are wide, resulting in poor frequency resolution (horizontal smearing).
- **Long Window:** Provides excellent frequency resolution, separating closely spaced pitches clearly. However, time resolution degrades, smearing transient events across a wide time region.

For fingerprinting, we settle on a middle ground (e.g.,  $N_{FFT} = 4096$ ) that balances localizing peaks in time while resolving musical pitches.

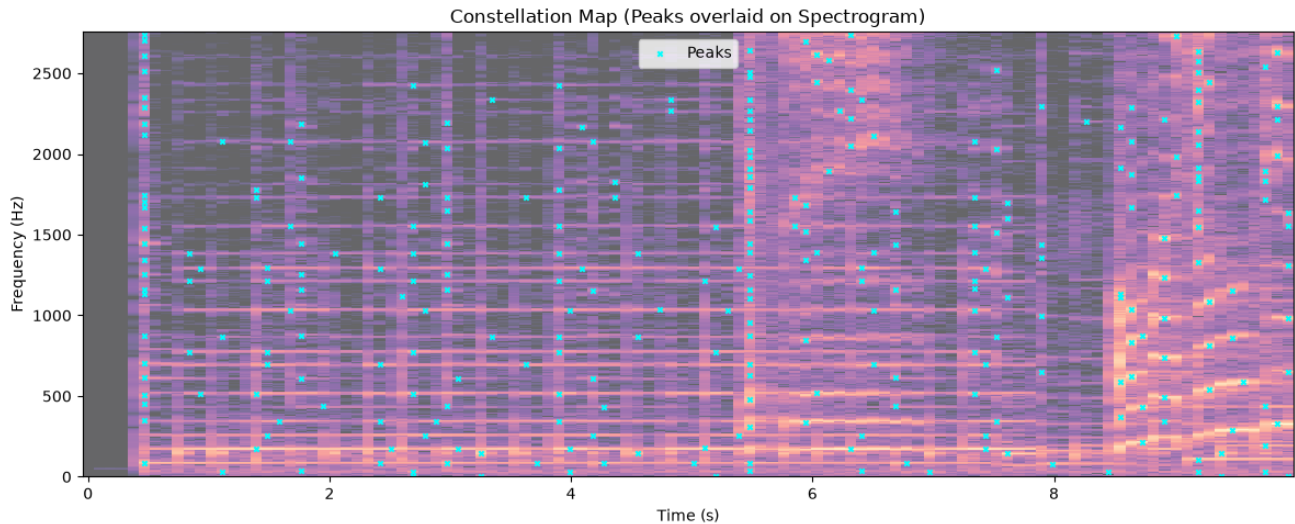


**Figure 2:** Comparison of short window (top) and long window (bottom) spectrograms, illustrating the time-frequency tradeoff.

### 3 3. Constellation Map (Peak Picking)

The fingerprint relies on a sparse set of standout points rather than the entire dense spectrogram. We extract local maxima (peaks) from the spectrogram magnitude. A peak is defined as a point that is louder than its immediate neighbors in a small 2D time-frequency window, and above a minimum amplitude threshold (to ignore background noise).

Figure 3 shows the extracted constellation map overlaid on the spectrogram. Extracting local maxima ensures peaks are spread across the entire clip's structural features, rather than clumping around a single globally loud event.



**Figure 3:** Constellation map: local maxima (cyan crosses) overlaid on the spectrogram.

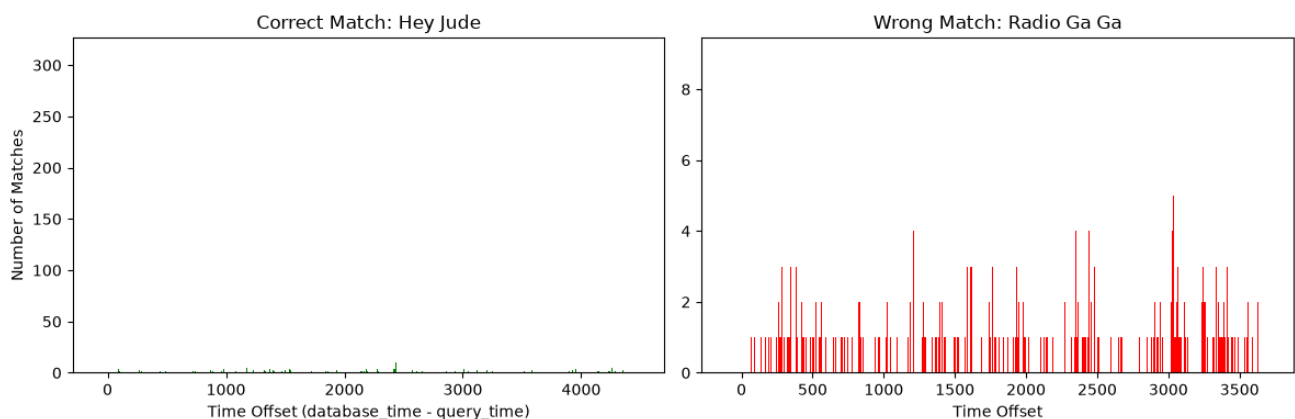
## 4 Hash Generation and Pairing

A single peak ( $time, frequency$ ) is not highly unique across a large database of songs. To create a robust signature, we pair nearby peaks into compact hashes:  $(f_{anchor}, f_{target}, \Delta t)$ , where  $\Delta t$  is the time gap between the anchor peak and target peak. This combinatorial pairing creates highly specific fingerprints that dramatically reduce false collisions.

## 5 Identification and Offset Matching

To identify a query clip, we fingerprint it to extract hashes and look up those hashes in our database. For every matching hash, we compute the relative time offset between the database occurrence and the query occurrence:  $Offset = t_{database} - t_{query}$ .

Figure 4 shows the offset histograms for a correct match versus an incorrect candidate. Because the query is a time-shifted excerpt of the true song, all correct hash matches vote for the *same* time offset, creating a massive, sharp spike. Conversely, random hash collisions with an incorrect song scatter randomly across many offsets.

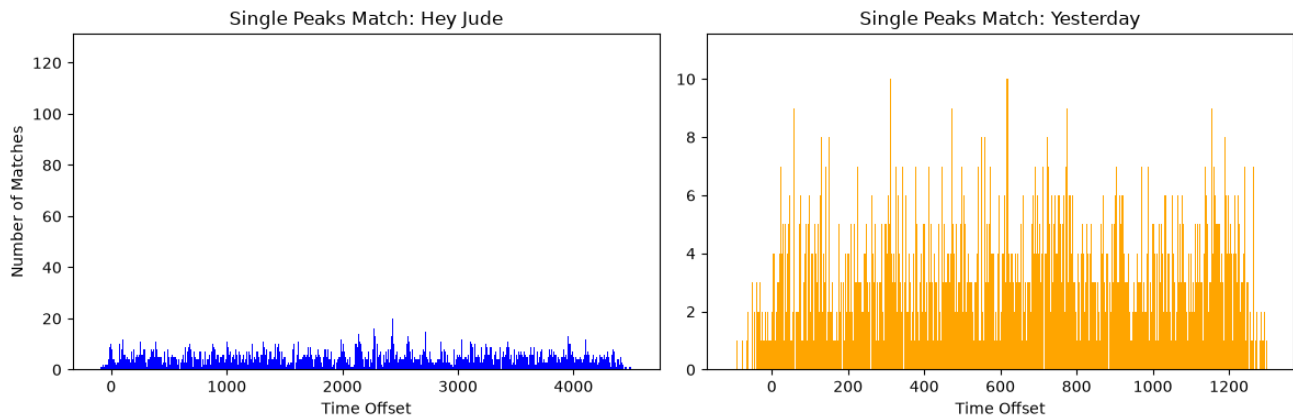


**Figure 4:** Offset histograms for paired hashes. The correct match (left) shows a decisive spike at a consistent offset. The wrong match (right) shows scattered, random noise.

## 6 Paired Hashes vs. Single Peaks

We repeated the identification process using only single-peak matching (e.g., matching just the frequency bin). As shown in Figure 5, single peaks yield a much flatter histogram with high background noise. The combinatorics of pairing peaks

$(f_1, f_2, \Delta t)$  forces matches to possess rigid structural relationships, making the correct song's spike vastly more decisive compared to background collision noise.

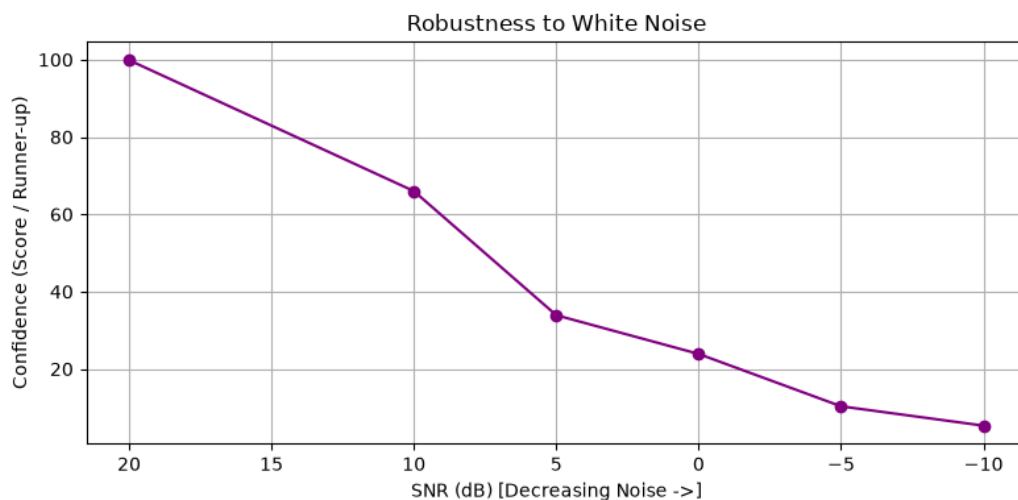


**Figure 5:** Offset histograms using single-peak matching. The spike is less decisive and the noise floor is much higher.

## 7 7. Robustness Testing

### 7.1 a. Noise Robustness

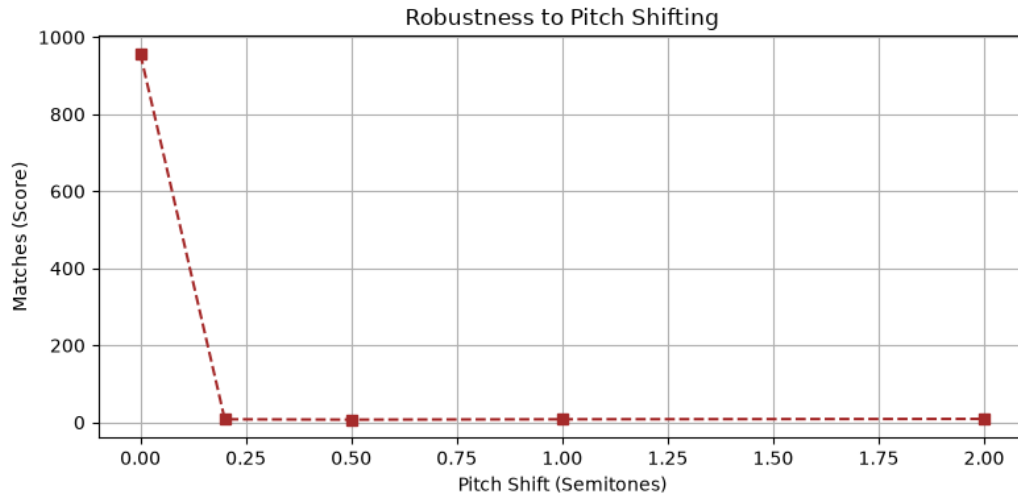
We added white Gaussian noise at decreasing Signal-to-Noise Ratios (SNR) to a query clip. Figure 6 shows that the system remains highly confident even under moderate noise, failing only when the noise entirely buries the dominant structural peaks (e.g., below 0 dB SNR).



**Figure 6:** Identification confidence as a function of SNR.

### 7.2 b. Pitch Shift Robustness

We artificially shifted the pitch of the query clip by fractions of a semitone. As Figure 7 shows, even a tiny shift ( $\geq 0.5$  semitones) breaks the identifier completely. Although the song sounds perceptually identical to humans, our database hashes rely on *exact* absolute frequency bins. Shifting the pitch moves all peaks vertically out of their recorded bins, causing total hash lookup failure.



**Figure 7:** Identification score as a function of artificial pitch shifting.

### 7.3 c. Proposed Improvements

To make the system robust to slight pitch variations, one concrete change would be to use **frequency ratios** ( $f_{target}/f_{anchor}$ ) or logarithmic frequency gaps instead of absolute frequencies in the hash tuple. Because pitch shifting moves all frequencies by a constant multiplicative factor, their ratios remain invariant, allowing hashes to match despite pitch distortions. Alternatively, we could perform **fuzzy matching** by quantizing frequency bins with a larger tolerance.